

Professional AC Core Cutting Business Website

System Architecture Document • v1.0 • Production Baseline

AREA	ARCHITECTURE DECISION
Product	Mobile-first local-service lead-generation website
Frontend	Next.js + TypeScript + Tailwind CSS
Rendering	Server-rendered / statically generated public pages; minimal client JavaScript
Backend	Next.js server-side/API layer for validation and lead handling
Database	PostgreSQL relational database
Media	Optimized responsive images; public asset/media storage
Integrations	Phone, WhatsApp, map/review links, optional email/CRM/analytics
Deployment	Local → Preview/Staging → Production
Primary outcome	Qualified calls, WhatsApp enquiries and quote requests

Architecture principle: keep the public website fast and SEO-friendly, keep secrets and database access server-side, and make the lead path reliable with graceful failure handling.

This document converts the approved product/technical requirements into a concrete system-level architecture. PostgreSQL, UUIDs and specific infrastructure choices are implementation recommendations where the source requirements do not mandate them.

1. Architecture Goals

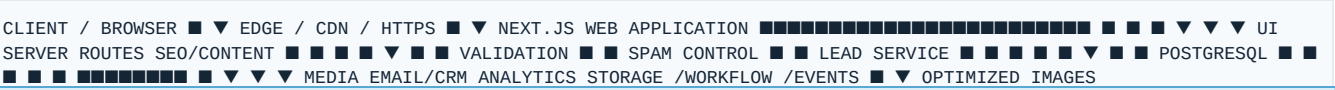
- Fast mobile-first experience for local customers.
- Strong local SEO through server-rendered/static public content.
- Reliable Call, WhatsApp and Request Quote conversion paths.
- Secure server-side enquiry handling with validation and anti-spam controls.
- Reusable content-driven architecture for services, gallery, reviews, FAQs and service areas.
- Minimal client-side JavaScript and restrained third-party dependencies.
- Clear separation between presentation, application logic, data and external integrations.
- Easy progression from local development to staging and production.
- Extensible foundation for future scheduling, lead dashboard, WhatsApp workflows, multilingual content and quote tools.

Architecture quality attributes

QUALITY	TARGET	ARCHITECTURAL RESPONSE
Performance	Strong Core Web Vitals	SSR/SSG, optimized images, limited fonts/JS, caching
SEO	Indexable local pages	Server-rendered public routes, metadata, canonical, sitemap, schema
Security	No secret exposure	Server-only credentials, validation, sanitization, rate limiting
Availability	Lead path remains usable	Graceful errors; direct phone/WhatsApp fallbacks
Accessibility	Usable keyboard/mobile assistive tech	Semantic HTML, labels, focus states, contrast
Maintainability	Reusable production code	Typed components, content model, repositories/services
Scalability	Support future growth	Modular data model and isolated integrations

2. High-Level System Architecture

The recommended system is a layered server-rendered web application. Public content is read through the server/data layer; quote submissions pass through a protected application endpoint before persistence and optional lead delivery.



External services are adapters, not core business logic. If email, CRM, maps or analytics changes, the application should not require a rewrite of the core domain model.

3. Layered Architecture

LAYER	RESPONSIBILITY	EXAMPLES
Presentation	Render accessible UI and user interactions	App Router pages, components, forms, CTA buttons
Application	Orchestrate use cases and business rules	Enquiry service, content queries, analytics event service
Domain / Types	Define stable business entities and contracts	Service, Review, FAQ, Enquiry, ServiceArea types
Data Access	Execute safe database operations	Repositories, parameterized queries, ORM
Infrastructure	Connect to external/runtime services	PostgreSQL client, email provider, media/CDN, analytics
Configuration	Environment-specific values	Public config + server-only secrets

Dependency direction

UI → application use case → repository/data access → database
UI → application adapter → external service
Configuration → infrastructure; secrets never flow into browser bundles

The architecture avoids having UI components directly query PostgreSQL or directly contain provider credentials.

4. Route & Rendering Architecture

ROUTE GROUP	RENDERING	PURPOSE
/	SSR/SSG	Primary conversion landing page
/services	SSR/SSG	Service discovery
/services/[service]	SSR/SSG	Indexable service detail pages
/about	SSR/SSG	Business trust/story
/gallery	SSR/SSG	Real work proof
/reviews	SSR/SSG	Social proof
/faq	SSR/SSG	Customer questions
/service-areas	SSR/SSG	Local coverage
/contact	SSR/SSG + server action/API	Contact and enquiry conversion

The approved route inventory includes dedicated pages for AC Core Cutting, RCC Core Cutting, AC Drain Hole, Concrete Wall Drilling, Pipe & Cable Passage and Other Services.

Rendering rule

Public, indexable content should default to server rendering or static generation. Interactive behavior should be client-side only when needed, such as the mobile menu, FAQ accordion, gallery interaction and form submission states.

5. Frontend Component Architecture

```
src/ ■■■ app/ ■ ■■■ page.tsx ■ ■■■ services/ ■ ■■■ about/ ■ ■■■ gallery/ ■ ■■■ reviews/ ■ ■■■ faq/ ■ ■■■  
service-areas/ ■ ■■■ contact/ ■■■ components/ ■ ■■■ layout/ ■ ■■■ navigation/ ■ ■■■ hero/ ■ ■■■ services/ ■ ■■■  
trust/ ■ ■■■ gallery/ ■ ■■■ reviews/ ■ ■■■ faq/ ■ ■■■ contact/ ■ ■■■ ui/ ■■■ content/ ■■■ lib/ ■ ■■■ db/ ■ ■■■  
validation/ ■ ■■■ leads/ ■ ■■■ seo/ ■ ■■■ analytics/ ■■■ types/ ■■■ styles/ ■■■ public/ ■■■ images/ ■■■ icons/
```

Component rule

Reusable components own presentation and interaction; application services own business logic; repositories own persistence. Avoid duplicating service-card, CTA, validation or SEO behavior across pages.

6. Backend / Application Architecture

MODULE	RESPONSIBILITY
Content service	Fetch published services, FAQs, gallery, reviews and service areas
Business profile service	Provide verified business/contact data
Enquiry service	Validate, normalize, persist and route quote requests
Validation module	Shared server-side schemas and field constraints
Spam protection	Rate limiting + honeypot/CAPTCHA-equivalent where required
SEO module	Metadata, canonical URLs, sitemap, robots and structured data
Analytics module	Allowlisted conversion events with minimal metadata
Integration adapters	Email/CRM, maps/review links and other external services

API boundary

Recommended endpoint: `POST /api/enquiries`

Optional future endpoints can remain server-only or protected by authentication. Public content does not need a general CRUD API for the MVP.

The technical requirements explicitly call for a POST enquiry flow over HTTPS with server validation, spam controls, safe errors and a lead destination such as email/CRM/workflow.

7. Enquiry / Lead Architecture

The enquiry path is the most business-critical backend flow.

CUSTOMER → QUOTE FORM → HTTPS POST → SERVER VALIDATION → NORMALIZATION → RATE LIMIT / BOT CHECK → SERVICE/AREA CONTEXT → DATABASE INSERT → LEAD DELIVERY → SUCCESS RESPONSE

FAILURE	USER EXPERIENCE	SYSTEM BEHAVIOR
Validation error	Inline field guidance	Do not persist invalid lead
Spam/rate limit	Safe generic retry message	Reject/throttle; avoid revealing anti-abuse logic
Database failure	Generic unavailable message	Operational log; do not expose DB details
Email/CRM failure	Lead saved if possible	Queue/retry or surface operational alert
Network interruption	Form remains recoverable	Prevent duplicate submissions where feasible
WhatsApp/phone unavailable	Alternative CTA remains	Contact form remains primary fallback

Lead data

name, phone, WhatsApp preference, service type, location and message are the core requested fields. source_page and status are recommended operational additions.

8. Database Integration Architecture

WEB APP ■ ■■■ Content Repository ■■→ PostgreSQL ■ ■■■ business_profile ■ ■■■ services ■ ■■■ service_areas ■ ■■■ gallery_items ■ ■■■ reviews ■ ■■■ faqs ■ ■■■ Enquiry Repository ■■→ PostgreSQL ■ ■■■ enquiries ■ ■■■ Analytics Repository → PostgreSQL / analytics provider ■■■ analytics_events

The separate Database Schema Document defines the relational model and indexes. This architecture document focuses on how that database participates in the application rather than duplicating its full DDL.

Connection rules

- Database connection is server-side only.
- Use pooled connections where supported by deployment infrastructure.
- Use parameterized queries or a safe typed ORM.
- Keep migrations in source control.
- Use separate credentials/databases for local, staging and production.

9. Media & Asset Architecture

Real project photography is a core trust asset. Images should be treated as content infrastructure rather than arbitrary page decorations.

ASSET	ARCHITECTURE
Source	Owner-approved real work/team/business photos
Storage	Versioned public media storage or deployment asset system
Delivery	Responsive images through the framework/CDN where available
Formats	Modern optimized formats with appropriate fallback
Metadata	Alt text required for informative images
Naming	Stable non-sensitive filenames
Caching	Long-lived caching for immutable/versioned assets

Avoid loading large images before they are needed. Third-party maps and widgets should not block initial rendering.

10. SEO Architecture

SEO is built into the page architecture, not added after UI completion.

ELEMENT	SYSTEM RESPONSIBILITY
Title/description	Unique metadata per indexable route
Canonical	Stable canonical URL for each public page
Sitemap	Generated from published public routes/content
Robots	Allow public content; block internal/error/private routes
Structured data	LocalBusiness + Service using verified information
Internal links	Home ↔ services ↔ service areas ↔ contact
Images	Descriptive alt text + optimized media
Social metadata	Open Graph/social sharing metadata
Performance	Rendering and asset strategy supports Core Web Vitals

No location page or structured-data claim should be generated unless it reflects genuine service coverage and verified business information.

11. Security Architecture

BROWSER ■ HTTPS ▼ EDGE ■ ▼ NEXT.JS SERVER ■ ■■ schema validation ■ ■■ sanitization / normalization ■ ■■ rate limiting / bot protection ■ ■■ authorization where required ■ ■■ safe error mapping ▼ DATABASE / EXTERNAL SERVICES

- TLS/HTTPS in production.
- Never expose DB credentials, API keys or private integration secrets to client code.
- Server-side validation is authoritative even when client validation exists.
- Apply secure headers and appropriate cookie policies if cookies are introduced.
- Restrict administrative credentials and infrastructure access.
- Keep logs useful but minimize personal data.
- Patch dependencies and review build/security warnings before release.

12. Privacy & Data Architecture

DATA CLASS	HANDLING
Public business content	Owner-approved and intentionally public
Lead personal data	Collect only what is needed to respond
Analytics	Allowlisted events; avoid sensitive metadata
Logs	Operational diagnostics without raw personal content where possible
Backups	Protected, access-controlled and retention-limited
Staging	No real customer leads unless explicitly protected and required

The project is not designed as a customer-account system. No login/password table is required for the MVP.

13. Third-Party Integration Architecture

INTEGRATION	ROLE	RULE
Phone	Direct conversion	Use tel: CTA; no server dependency
WhatsApp	Direct conversion	Use approved business number/link; no secret required client-side
Google Maps	Location context	Lazy-load or link out; never block critical rendering
Google reviews/profile	Trust/public review path	Link to verified public profile
Email/CRM	Lead delivery	Server-side credentials; retry/failure handling
Analytics	Conversion measurement	Privacy-conscious configuration; allowlisted events

The website should remain usable if a third-party widget fails. Direct Call, WhatsApp and the enquiry form are the core conversion mechanisms.

14. Caching & Performance Architecture

Recommended caching hierarchy:

```
STATIC ASSETS → CDN / LONG CACHE  
PUBLIC CONTENT → ISR/STATIC CACHE WHERE SUITABLE  
BUSINESS DATA → SERVER CACHE WITH SAFE REVALIDATION  
ENQUIRIES → NO PUBLIC CACHE
```

- Optimize images before delivery and use responsive sizing.
- Limit font families/weights.
- Keep client JavaScript small.
- Lazy-load non-critical images and third-party embeds.
- Do not block initial rendering on maps or analytics.
- Use framework caching/revalidation for public content when appropriate.
- Keep enquiry POST responses uncached.

15. Observability & Analytics Architecture

Operational monitoring and conversion analytics have different purposes.

SYSTEM	MEASURES
Application monitoring	Errors, failed requests, latency, deployment health
Lead monitoring	Enquiry inserts, delivery failures, status distribution
Conversion analytics	phone_click, whatsapp_click, quote_start, quote_submit, map_click, service_cta_click
Performance	Core Web Vitals, asset size, server response time
SEO health	Sitemap, indexing, structured-data and broken-link checks

Do not rely on analytics to determine whether a lead was successfully persisted. The database/application transaction is the source of truth for enquiries.

16. Deployment Architecture

DEVELOPER ■ ▼ LOCAL ENVIRONMENT ■ git push / PR ▼ PREVIEW / STAGING ■ QA + approval ▼ PRODUCTION ■■■ HTTPS / custom domain ■■■ Next.js application ■■■ PostgreSQL ■■■ media/CDN ■■■ monitoring ■■■ analytics

ENV	PURPOSE	DATA RULE
Local	Development	Synthetic/demo data
Preview/Staging	Integration + QA	Separate DB; no accidental production data
Production	Real customer traffic/leads	Protected credentials, backups, monitoring

17. Reliability & Recovery

- Primary conversion routes must have independent fallbacks: Call, WhatsApp and quote form.
- Form submission failures must return safe, actionable messages.
- Lead persistence should be attempted before non-critical analytics work.
- External delivery failures should be observable and retryable where supported.
- Database backups should be configured before production launch.
- Deployment should include smoke tests for home, service detail, contact and enquiry submission.
- Schema migrations should avoid destructive changes without a tested recovery path.

Graceful degradation

If analytics fails → website still works.

If map fails → address/service-area content and external map link still work.

If CRM/email temporarily fails → enquiry persistence should remain the authoritative lead record when possible.

If JavaScript fails → core public content and direct contact paths remain usable.

18. Architecture Security Boundaries

TRUST BOUNDARY [UNTRUSTED] Browser input / third-party traffic ■ ▼ [PROTECTED SERVER BOUNDARY] Validation → Rate limit
→ Bot controls → Business rules ■ ▼ [DATA BOUNDARY] PostgreSQL / server integrations ■ ▼ [ADMIN / OPERATIONS]
Restricted credentials + monitoring

The browser is never treated as trusted. Client-side validation is a UX enhancement only; server-side validation is mandatory.

19. Future Extension Architecture

FUTURE FEATURE	EXTENSION PATH
Lead dashboard	Add authenticated admin layer + lead views around enquiries
Scheduling	Add availability/booking entities without changing service content
WhatsApp workflows	Add server-side messaging adapter + delivery state
Multilingual English/Gujarati/Hindi	Add locale-aware content model and route strategy
Quote calculator	Add rules/estimation module; keep pricing logic separate
Blog/SEO content	Add posts/categories/content metadata
Review automation	Add integration/sync layer with source attribution
CMS	Add controlled content administration over existing entities

These are future extensions, not MVP requirements. The architecture is intentionally modular so they can be introduced without turning the public website into an unnecessarily complex platform.

20. Architecture Acceptance Criteria

- All approved public routes render correctly on mobile and desktop.
- Public pages are server-rendered or statically generated where appropriate.
- Client JavaScript is limited to genuinely interactive behavior.
- Service, gallery, review, FAQ and service-area content has a clear data source.
- Quote submissions use HTTPS and server-side validation.
- Spam controls protect the enquiry endpoint.
- Database access is server-only and parameterized/safely abstracted.
- Secrets are never included in browser bundles.
- Phone and WhatsApp paths work independently of non-critical integrations.
- Maps/analytics do not block critical page rendering.
- SEO metadata, canonical, sitemap, robots and structured data are generated correctly.
- Production has HTTPS, backups, monitoring and environment separation.
- Accessibility, performance, security and cross-browser QA are completed before launch.

21. Final System Architecture Lock

The production baseline is:

NEXT.JS + TYPESCRIPT ■ ■■■ SERVER-RENDERED PUBLIC EXPERIENCE ■ ■■■ Home ■ ■■■ Services ■ ■■■ Service Details ■ ■■■ About ■ ■■■ Gallery ■ ■■■ Reviews ■ ■■■ FAQ ■ ■■■ Service Areas ■ ■■■ Contact ■ ■■■ SERVER APPLICATION LAYER ■ ■■■ Content ■ ■■■ Enquiry ■ ■■■ Validation ■ ■■■ SEO ■ ■■■ Analytics ■ ■■■ POSTGRESQL ■ ■■■ Business ■ ■■■ Services ■ ■■■ Areas ■ ■■■ Gallery ■ ■■■ Reviews ■ ■■■ FAQs ■ ■■■ Enquiries / Events ■ ■■■ EXTERNAL ADAPTERS ■ ■■■ Email / CRM ■ ■■■ WhatsApp / Phone ■ ■■■ Maps / Reviews ■ ■■■ Analytics

**MASTER FLOW: DISCOVER → TRUST → EVALUATE → CONTACT → VALIDATE → PERSIST
LEAD → BUSINESS FOLLOW-UP**

This architecture is the implementation bridge between the UI/UX specification, App Flow specification, Backend Schema specification and Database Schema specification.

SYSTEM ARCHITECTURE DOCUMENT v1.0 • Production Baseline